

环境搭建

(参考<https://www.cnblogs.com/gatsby123/p/9746193.html>)

```
~/oslab$ git clone https://pdos.csail.mit.edu/6.828/2018/jos.git lab
~/oslab$ gcc -m32 -print-libgcc-file-name
/usr/lib/gcc/x86_64-linux-gnu/version/32/libgcc.a
~/oslab$ git clone https://github.com/mit-pdos/6.828-qemu.git qemu
~/oslab$ cd qemu
~/oslab/qemu$ sudo apt-get install -y pkg-config zlib1g-dev libglib2.0-dev
libpixman-1-dev
~/oslab/qemu$ ./configure --disable-kvm --target-list="i386-softmmu x86_64-softmmu" --python=python2.7 --disable-werror
~/oslab/qemu$ make
```

出现报错

```
/usr/bin/ld: qga/commands-posix.o: in function `dev_major_minor':
/home/ubuntu/oslab/qemu/qga/commands-posix.c:633:(.text+0x23e): undefined
reference to `major'
/usr/bin/ld: /home/ubuntu/oslab/qemu/qga/commands-posix.c:634:(.text+0x24d):
undefined reference to `minor'
collect2: error: ld returned 1 exit status
make: *** [Makefile:288: qemu-ga] Error 1
```

在qga/commands-posix.o中添加以下行后make即可

```
#include <sys/types.h>
#include <sys/sysmacros.h>
```

```
~/oslab/qemu$ sudo make install
~/oslab/qemu$ cd ../lab
~/oslab/lab$ make
+ as kern/entry.S
+ cc kern/entrypgdir.c
+ cc kern/init.c
+ cc kern/console.c
+ cc kern/monitor.c
+ cc kern/printf.c
+ cc kern/kdebug.c
+ cc lib/printfmt.c
+ cc lib/readline.c
+ cc lib/string.c
+ ld obj/kern/kernel
+ as boot/boot.S
```

```
+ cc -Os boot/main.c
+ ld boot/boot
boot block is 396 bytes (max 510)
+ mk obj/kern/kernel.img
```

编译成功, warnings可以忽略

```
~/oslab/lab$ make qemu
sed "s/localhost:1234/localhost:26000/" < .gdbinit.tmpl > .gdbinit
qemu-system-i386 -drive file=obj/kern/kernel.img,index=0,media=disk,format=raw -
serial mon:stdio -gdb tcp::26000 -D qemu.log
VNC server running on `127.0.0.1:5900'
6828 decimal is XXX octal!
entering test_backtrace 5
entering test_backtrace 4
entering test_backtrace 3
entering test_backtrace 2
entering test_backtrace 1
entering test_backtrace 0
leaving test_backtrace 0
leaving test_backtrace 1
leaving test_backtrace 2
leaving test_backtrace 3
leaving test_backtrace 4
leaving test_backtrace 5
Welcome to the JOS kernel monitor!
Type 'help' for a list of commands.
K>
```

环境搭建结束

Lab 1: Booting a PC

Part 1: PC Bootstrap

(参考<https://www.cnblogs.com/gatsby123/p/9759153.html>)

1. getting familiarized with x86 assembly language, the QEMU x86 emulator, and the PC's power-on bootstrap procedure 简单介绍了汇编语言, 物理内存地址空间, BIOS
2. examine the boot loader for our 6.828 kernel ([lab/boot/](#)) 介绍了BIOS从磁盘0号扇区读取boot loader到0000:7c00处, 并将cs:ip设置成0000:7c00
3. delve into the initial template for our 6.828 kernel itself, named JOS ([kernel/](#)) 介绍进入内核后的一些操作

AT&T汇编语法

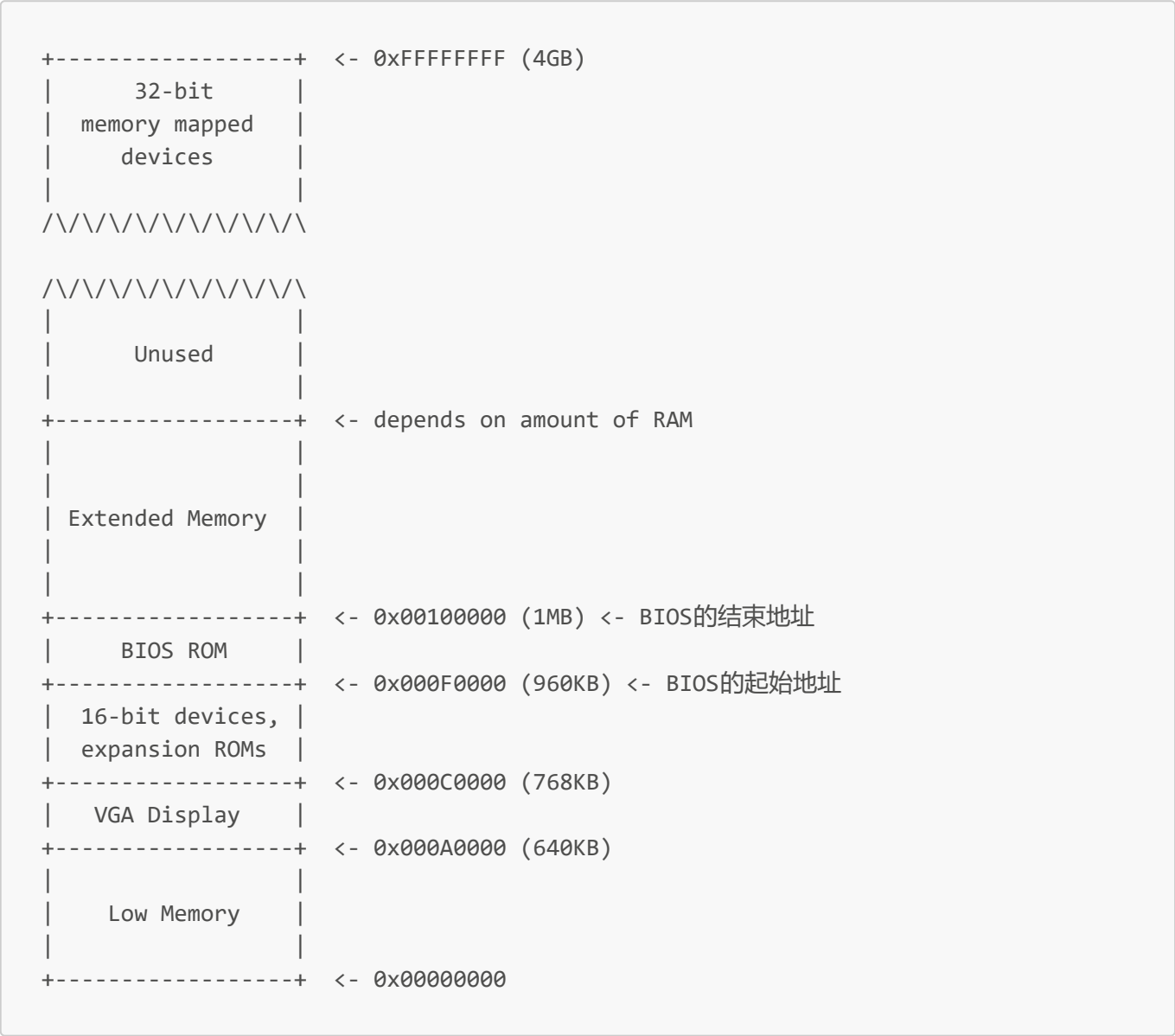
https://www.delorie.com/djgpp/doc/brennan/brennan_att_inline_djgpp.html

1. 寄存器使用%: `%eax`

- 2. 目的寄存器总是在左边
- 3. 立即数和常数使用\$: `$0xd00d`
- 4. `b`(byte), `w`(word), `l`(longword): `movw`
- 5. 访存`immed32(basepointer, indexpointer, indexscale)`的地址为`immed32 + basepointer + indexpointer * indexscale`

[Intel Architecture Manual](#)

Memory Layout



The ROM BIOS

In early PCs the BIOS was held in true **read-only memory (ROM)**, but current PCs store the BIOS in updateable flash memory.

使用GDB作为客户端，通过网络连接到QEMU中正在运行的 JOS 内核：

```
(shell 1) ~/oslab/lab/$ make qemu-gdb
(shell 2) ~/oslab/lab/$ make gdb
```

```
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
This GDB was configured as "x86_64-linux-gnu".
+ target remote localhost:26000
warning: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
warning: A handler for the OS ABI "GNU/Linux" is not built into this configuration
of GDB. Attempting to continue with the default i8086 settings.

The target architecture is set to "i8086".
[f000:fff0]    0xfffff0: ljmp    $0xf000,$0xe05b
0x0000ffff0 in ?? ()
```

这是 GDB 从远程目标 (QEMU) 读到的第一条指令，下面将逐项解释：

```
[f000:fff0]    0xfffff0: ljmp    $0xf000,$0xe05b
0x0000ffff0 in ?? ()
```

CS - Code Segment: CS 是代码段寄存器。在实模式下，它的值不是一个直接的内存地址，而是一个指向内存中某个段 (Segment) 的段基址 (Segment Base Address) 的指针。

IP - Instruction Pointer: IP 是指令指针寄存器 (在 32 位模式下被称为 EIP，64 位下为 RIP)。它可以看作是 当前代码段 (CS 指向的段) 内的一个偏移量 (Offset)。

```
[f000:fff0] -> CS = 0xf000 and IP = 0xffff0
physical address = 16 * segment + offset
                  = 0xf0000 + 0xffff0
                  = 0xfffff0 -> 16 bytes before the end of the BIOS (0x100000)

0xfffff0: ljmp    $0xf000,$0xe05b -> jmp backwards to an earlier location in the
BIOS
-> jumps to the segmented address CS = 0xf000 and IP = 0xe05b
-> physical address = 0xfe05b
-> 跳转到一个更低、更宽敞的地址 (0xfe05b)，那里有足够的连续空间来存放 BIOS 需要执行的大
量初始化代码 (检测硬件、建立中断向量表、扫描引导设备等)

0x0000ffff0 in ?? () -> 这是 GDB 对当前程序计数器 PC - Program Counter) 位置的另一种表
示。因为此时还没有加载任何符号信息，GDB 不知道这个地址对应什么函数，所以显示??
```

Run Step Instruction command in gdb to see what happens next:

```
(gdb) si
[f000:e05b]    0xfe05b: cml    $0x0,%cs:0x6ac8
0x0000e05b in ?? ()
(gdb) si
[f000:e062]    0xfe062: jne    0xfd2e1
0x0000e062 in ?? ()
```

```
(gdb) si
[f000:e066] 0xfe066: xor    %dx,%dx
0x0000e066 in ?? ()
(gdb) si
[f000:e068] 0xfe068: mov    %dx,%ss
0x0000e068 in ?? ()
(gdb) si
[f000:e06a] 0xfe06a: mov    $0x7000,%esp
0x0000e06a in ?? ()
```

```
cmpl    $0x0,%cs:0x6ac8 -> 比较物理地址0xf6ac8处的4字节值是否为0, 可能在初始化检查
jne     0xfd2e1 -> 如果不为0, 跳转到0xfd2e1
xor      %dx,%dx -> %dx置零
mov      %dx,%ss -> %ss置零, %ss是栈段寄存器(Stack Segment Register), 栈段的基地址被设置为 0x00000
mov      $0x7000,%esp -> %esp置为0x7000, %esp扩展栈指针寄存器(Extended Stack Pointer)
-> 当前的栈顶物理地址为: (SS << 4) + SP = (0x0000 << 4) + 0x7000 = 0x07000
-> 在物理内存的低地址区 (0x07000) 建立了一个栈, 任何push操作都会将数据写入低于0x07000的内存中
```

Part 2: The Boot Loader

Hard Drive Boot Mechanism

- **Sectors:** Floppy and hard disks for PCs are divided into 512 byte regions, the disk's minimum transfer granularity
- **Boot sector:** If the disk is bootable, the first sector is called the boot sector, this is where the boot loader code resides

When the BIOS finds a bootable floppy or hard disk, it loads the 512-byte boot sector into memory at physical addresses `0x7c00` through `0x7dff`, and then uses a `jmp` instruction to set the `CS:IP` to `0000:7c00`, passing control to the boot loader.

-> boot loader must fit into a measly 512 bytes

Boot Loader

Code: one assembly language source file, `boot/boot.S`, and one C source file, `boot/main.c`

1. switches the processor from real mode to **32-bit protected mode** to access all the memory above 1MB in the processor's physical address space, sections 1.2.7 and 1.2.8 of [PC Assembly Language](#)

-> translation of segmented addresses (segment:offset pairs) into physical addresses happens differently in protected mode

实模式下, CPU使用“段地址:偏移地址”的方式计算物理地址, 最大寻址范围为`0xFFFF:0xFFFF = 0x10FFEF` (约1MB+64KB)。实际上, 由于历史原因, 通常只能有效使用最低的1MB内存。

保护模式下，地址转换机制完全不同。通过全局描述符表（GDT），段寄存器不再直接存放段基址，而是作为一个“选择子（Selector）”，指向GDT中的一个表项。该表项包含了段的基地址（32位）、限制和访问权限。这使得CPU可以寻址到完整的4GB（ 2^{32} 字节）物理地址空间。现代计算机的内存远大于1MB，因此必须进入保护模式才能使用它们。

-> after the transition offsets are 32 bits instead of 16

2. reads the kernel from the hard disk by directly accessing the IDE disk device registers via the x86's special I/O instructions

IDE (Integrated Drive Electronics) 硬盘控制器使用的主要是端口映射I/O。这意味着控制器有一组独立的、不同于内存地址空间的端口地址（Port Addresses）。

Boot Loader的核心任务之一就是要把操作系统内核从慢速的磁盘加载到快速的内存中。它不能使用高级的操作系统API（因为OS还没运行），所以必须“亲自动手”，通过最底层的硬件接口——即直接读写磁盘控制器的设备寄存器——来完成这个任务。

`obj/boot/boot.asm`: a disassembly of the boot loader that our GNUmakefile creates after compiling the boot loader

`obj/kern/kernel.asm`: a disassembly of the JOS kernel

GDB commands

labguide

1. `b *0x7c00` sets a breakpoint at address `0x7c00`
2. `c` causes QEMU to continue execution until the next breakpoint (or until you press `Ctrl-C` in GDB)
3. `si N` steps through the instructions `N` at a time
4. `x/Ni ADDR` (`N` is the number of consecutive instructions to disassemble and `ADDR` is the memory address at which to start disassembling) examines instructions in memory

```
(gdb) b *0x7c00
Breakpoint 1 at 0x7c00
(gdb) c
Continuing.
[ 0:7c00] => 0x7c00: cli
Breakpoint 1, 0x00007c00 in ?? ()
(gdb) si
[ 0:7c01] => 0x7c01: cld
0x00007c01 in ?? ()
(gdb) si
[ 0:7c02] => 0x7c02: xor    %ax,%ax
0x00007c02 in ?? ()
```

-> 后面就是`boot/boot.S`里的汇编，说明BIOS已经将boot loader加载到`0x7c00`处并跳转到这里让boot loader take over了，可以对照着`boot/boot.S`和`obj/boot/boot.asm`来看

Questions

1. At what point does the processor start executing **32-bit code**? What exactly causes the switch from 16- to 32-bit mode?
2. What is the **last instruction of the boot loader** executed, and what is the **first instruction of the kernel** it just loaded?
3. Where is the **first instruction of the kernel**?
4. How does the boot loader decide **how many sectors** it must read in order to fetch the entire kernel from disk? Where does it find this information?

Analysis

`boot loader`指的是BIOS加载`kernel`的程序，反汇编在`obj/boot/boot.asm`中

由`boot/boot.S`和`boot/main.c`组成，目的是调用`kernel`和此前的准备工作（转换地址翻译模式等）

`bootloader (boot.S) -> bootmain (main.c)`

1. 加载全局描述符表寄存器(GDTR) `gdt_desc`标签处的数据包含了GDT的大小和起始地址，这条指令让CPU知道了我们定义的保护模式段表在哪里

```
%cr0 = $CR0_PE_ON = 0x1 (protected mode enable flag)
```

正式进入保护模式，但此时的内存寻址方式尚未更新，因为CS还是实模式的值

`boot/boot.S`

```
# Switch from real to protected mode, using a bootstrap GDT
# and segment translation that makes virtual addresses
# identical to their physical addresses, so that the
# effective memory map does not change during the switch.
lgdt    gdt_desc
movl    %cr0, %eax
orl     $CR0_PE_ON, %eax
movl    %eax, %cr0
```

2. 将`$PROT_MODE_CSEG(0x8)`加载到CS寄存器。这使CS现在作为一个选择子(index?)，指向GDT中定义的内核代码段

跳转到`protcseg`标签处的指令继续执行

彻底完成了到保护模式的切换，因为它更新了CS，CPU从此开始使用GDT中的描述符来翻译地址

```
# Jump to next instruction, but in 32-bit code segment.
# Switches processor into 32-bit mode.
ljmp    $PROT_MODE_CSEG, $protcseg

.code32                                # Assemble for 32-bit mode
```

obj/boot/boot.asm

```
7c1e:  0f 01 16          lgdtl  (%esi)
7c21:  64 7c 0f          fs jl  7c33 <protcseg+0x1>
7c24:  20 c0             and    %al,%al
7c26:  66 83 c8 01       or     $0x1,%ax
7c2a:  0f 22 c0          mov    %eax,%cr0
7c2d:  ea               .byte 0xea
7c2e:  32 7c 08 00       xor    0x0(%eax,%ecx,1),%bh
```

重置段指针后调用`bootmain`

`bootmain (main.c) -> kernel (kernel.asm)`

1. 从硬盘的第0扇区开始 (`offset = 0`)，读取4KB (`SECTSIZE*8 = 512*8 = 4096`) 的数据到内存地址 `0x10000`

boot/main.c

```
`#define ELFHDR      ((struct Elf *) 0x10000) // scratch space`

// read 1st page off disk
readseg((uint32_t) ELFHDR, SECTSIZE*8, 0);
```

2. 检查内存中ELFHDR位置的`e_magic`字段是否等于标准的ELF魔数

```
if (ELFHDR->e_magic != ELF_MAGIC)
    goto bad;
```

3. 遍历程序头表中的每一个条目，对于每个段：

`ph->p_pa`: 物理地址，该段应该被加载到的目标内存地址

`ph->p_memsz`: 该段在内存中的大小

`ph->p_offset`: 该段在ELF文件中的起始偏移量

程序头表是一个由多个`Proghdr`（程序头）结构组成的数组。每个`Proghdr`描述了一个内核中的段（**Segment**），例如代码段（`.text`）、数据段（`.data`）等。要遍历程序头表，首先要知道程序头表的位置和大小

`eph = ph + ELFHDR->e_phnum`;是终止循环的判断，所以`boot loader`是通过ELF文件头中的信息得知一些数目信息的。`Boot Loader`首先从磁盘的第一个扇区（紧跟在`boot loader`自身之后）读取足够的数据到内存中的`0x10000`（见1.）。这部分数据包含了内核映像的开头，其中就包括ELF文件头

ELF文件头就像一个文件的“总目录”，它本身并不包含代码数据，但它指明了在哪里可以找到真正的“地图”。其中两个关键字段是：

`e_phoff`: 程序头表 (Program Header Table) 在文件中的偏移量 (字节为单位)

`e_phnum`: 程序头表中一共有多少个条目

通过这两个值, `boot loader` 就能找到并遍历程序头表

对于每一个段, `Proghdr` 提供了将其从磁盘加载到内存所需的全部信息: `ph->p_pa`, `ph->p_memsz`, `ph->p_offset`

ELF -> Proghdr -> Section (.text, .rodata, .data, .bss)

```
// load each program segment (ignores ph flags)
ph = (struct Proghdr *) ((uint8_t *) ELFHDR + ELFHDR->e_phoff);
eph = ph + ELFHDR->e_phnum;
for (; ph < eph; ph++)
    readseg(ph->p_pa, ph->p_memsz, ph->p_offset);
```

`Boot Loader` 决定读取多少扇区的过程: 先读一点 (ELF头): 读取磁盘开头的少量扇区, 获取ELF头; 根据ELF头提供的信息, 找到程序头表; 遍历程序头表的每一个条目, 每个条目告诉 `boot loader` 一个段的大小和位置。 `Boot Loader` 于是读取每个 `segment (readseg)` 所在的 `sector (readsect)`。

4. 跳转到内核入口

`ELFHDR->e_entry` 是ELF头中指定的内核的入口点地址 (一个内存地址)

这行代码将该地址强制转换为一个无参数、无返回值的函数指针, 然后调用这个函数

这是 `boot loader` 和 `kernel` 的转换点

```
((void (*)(void)) (ELFHDR->e_entry))();
```

`bad` 让程序挂起

```
bad:
    outw(0x8A00, 0x8A00);
    outw(0x8A00, 0x8E00);
    while (1)
        /* do nothing */;
```

对应的反汇编代码:

```
obj/boot/boot.asm

7d58:  73 17                jae    7d71 <bootmain+0x58>
7d71:  ff 15 18 00 01 00    call   *0x10018
```

```
-> 循环条件不满足时跳出，跳转到kernel入口
-> 后面由boot.asm转到kernel.asm
```

```
(gdb) b *0x7d71
Breakpoint 3 at 0x7d71
(gdb) c
Continuing.
=> 0x7d71:      call    *0x10018
    -> boot loader执行的最后一条指令
Breakpoint 3, 0x00007d71 in ?? ()
(gdb) si
=> 0x10000c:     movw    $0x1234,0x472
    -> kernel执行的第一条指令
    -> 对应kernel.asm中
    -> f010000c:    66 c7 05 72 04 00 00      movw    $0x1234,0x472
0x0010000c in ?? ()
```

```
~/oslab$ gcc -g -o pointers pointers.c
~/oslab$ ./pointers
1: a = 0x7ffdfd603680, b = 0x5c6403c702a0, c = (nil)
2: a[0] = 200, a[1] = 101, a[2] = 102, a[3] = 103
3: a[0] = 200, a[1] = 300, a[2] = 301, a[3] = 302
4: a[0] = 200, a[1] = 400, a[2] = 301, a[3] = 302
5: a[0] = 200, a[1] = 128144, a[2] = 256, a[3] = 302
6: a = 0x7ffdfd603680, b = 0x7ffdfd603684, c = 0x7ffdfd603681
~/oslab$ gdb pointers
TODO: 补充指针操作的完整过程
```

ELF binary - Executable and Linkable Format

ELF: executable to be a header with loading information, followed by several program sections, each of which is a contiguous chunk of code or data intended to be loaded into memory at a specified address. The boot loader does not modify the code or data; it loads it into memory and starts executing it.

当使用gcc编译一个C程序时，生成的可执行文件（如obj/kern/kernel）通常就是ELF格式的

ELF由下面3部分组成：

1. ELF头包含程序头表（**Program Header Table**）的位置和大小
2. 程序头表由一个或多个程序头（**Program Header**）条目组成，每个条目描述了一个段（Segment），也就是一块需要被加载到内存中的连续数据
3. 段包含里面是什么（是代码还是数据？），应该被送到内存的哪个地址（**p_pa**，物理地址），在文件中的位置（**p_offset**，偏移量），内存中的大小（**p_memsz**）

```
~/oslab/lab$ objdump -h obj/kern/kernel
```

```
obj/kern/kernel:      file format elf32-i386

Sections:
Idx Name              Size      VMA          LMA          File off  Algn
  0 .text              00001a16  f0100000  00100000  00001000  2**4
                        CONTENTS, ALLOC, LOAD, READONLY, CODE
  1 .rodata            000006b0  f0101a20  00101a20  00002a20  2**5
                        CONTENTS, ALLOC, LOAD, READONLY, DATA
  2 .stab              00000001  f01020d0  001020d0  000030d0  2**0
                        CONTENTS, ALLOC, LOAD, DATA
  3 .stabstr           00000001  f01020d1  001020d1  000030d1  2**0
                        CONTENTS, ALLOC, LOAD, DATA
  4 .data              00009300  f0103000  00103000  00004000  2**12
                        CONTENTS, ALLOC, LOAD, DATA
  5 .got               00000008  f010c300  0010c300  0000d300  2**2
                        CONTENTS, ALLOC, LOAD, DATA
  6 .got.plt           0000000c  f010c308  0010c308  0000d308  2**2
                        CONTENTS, ALLOC, LOAD, DATA
  7 .data.rel.local    00001000  f010d000  0010d000  0000e000  2**12
                        CONTENTS, ALLOC, LOAD, DATA
  8 .data.rel.ro.local 00000044  f010e000  0010e000  0000f000  2**2
                        CONTENTS, ALLOC, LOAD, DATA
  9 .bss               00000661  f010e060  0010e060  0000f060  2**5
                        CONTENTS, ALLOC, LOAD, DATA
 10 .comment           0000002b  00000000  00000000  0000f6c1  2**0
                        CONTENTS, READONLY
```

Load Address & Link Address

LMA - Load Memory Address

The load address of a section is the memory address at which that section should be loaded into memory. 代码实际被加载器放置在物理内存中的地址。编译时由链接器决定，并固化在二进制文件中。决定了代码中绝对地址的值。

VMA - Virtual Memory Address (or link address)

The link address of a section is the memory address from which the section expects to execute. 代码被编译链接时假定的运行地址。加载时由boot loader决定。链接地址（VMA）是一个编译时概念。它由链接器嵌入到二进制文件中，告诉代码：“你被设计为在地址 X 运行”。

Typically, the link and load addresses are the same.

```
~/oslab/lab$ objdump -h obj/boot/boot.out

obj/boot/boot.out:      file format elf32-i386

Sections:
Idx Name              Size      VMA          LMA          File off  Algn
  0 .text              0000018c  00007c00  00007c00  00000074  2**2
                        CONTENTS, ALLOC, LOAD, CODE
```

```

1 .eh_frame      0000009c 00007d8c 00007d8c 00000200 2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
2 .comment       0000002b 00000000 00000000 0000029c 2**0
                  CONTENTS, READONLY

```

The BIOS loads the boot sector into memory starting at address `0x7c00`, so this is the boot sector's load address.

We set the link address by passing `-Ttext 0x7C00` to the linker in `boot/Makefrag`.

```
boot/Makefrag
```

```
$(V)$(LD) $(LDFLAGS) -N -e start -Ttext 0x7C00 -o $@.out $^
```

```
-> $(V)$(LD) $(LDFLAGS) -N -e start -Ttext 0x7C01 -o $@.out $^
```

```

(gdb) b *0x7c00
Breakpoint 1 at 0x7c00
(gdb) c
Continuing.
[ 0:7c00] => 0x7c00: xchg    %eax,%eax

Breakpoint 1, 0x00007c00 in ?? ()
(gdb) si
[ 0:7c02] => 0x7c02: nop
0x00007c02 in ?? ()
(gdb) si
[ 0:7c03] => 0x7c03: cli    <- boot sector
0x00007c03 in ?? ()
(gdb) si
[ 0:7c04] => 0x7c04: cld
0x00007c04 in ?? ()
(gdb) si
[ 0:7c05] => 0x7c05: xor     %ax,%ax
0x00007c05 in ?? ()
(gdb) si
[ 0:7c07] => 0x7c07: mov     %ax,%ds

```

`start-0x3`处的两条指令是由链接器插入的填充或对齐指令，所以boot sector在`0x7c04`处开始

```
obj/boot/boot.asm
```

```

00007c01 <start-0x3>:
    7c01:  66 90                xchg    %ax,%ax
    7c03:  90                   nop

00007c04 <start>:

```

```
7c04:  fa          cli
7c05:  fc          cld
7c06:  31 c0       xor    %eax,%eax
7c08:  8e d8       mov    %eax,%ds
7c0a:  8e c0       mov    %eax,%es
7c0c:  8e d0       mov    %eax,%ss
```

Load Address和Link Address不一致导致

- 1. BIOS将引导扇区的512字节数据从磁盘加载到物理内存地址0x7c00到0x7dff的区域
- 2. BIOS执行jmp 0x7c00，将CPU的控制权交给boot loader的第一个字节
- 3. 链接器在解析代码中的所有绝对地址时，都基于一个假设：代码的起始点在0x7c04
- 4. 绝对地址引用导致崩溃

The kernel is telling the boot loader to load it into memory at a low address (1 megabyte), but it expects to execute from a high address.

```
~/oslab/lab$ objdump -x obj/kern/kernel

obj/kern/kernel:      file format elf32-i386
obj/kern/kernel
architecture: i386, flags 0x00000112:
EXEC_P, HAS_SYMS, D_PAGED
start address 0x0010000c

Program Header:
  LOAD off    0x00001000 vaddr 0xf0100000 paddr 0x00100000 align 2**12
    filesz 0x000020d2 memsz 0x000020d2 flags rwx
  LOAD off    0x00004000 vaddr 0xf0103000 paddr 0x00103000 align 2**12
    filesz 0x0000b6c1 memsz 0x0000b6c1 flags rw-
  STACK off   0x00000000 vaddr 0x00000000 paddr 0x00000000 align 2**4
    filesz 0x00000000 memsz 0x00000000 flags rwx

Sections:
Idx Name          Size      VMA          LMA          File off  Algn
  0 .text          00001a16 f0100000 00100000 00001000  2**4
    CONTENTS, ALLOC, LOAD, READONLY, CODE
  1 .rodata         000006b0 f0101a20 00101a20 00002a20  2**5
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  2 .stab           00000001 f01020d0 001020d0 000030d0  2**0
    CONTENTS, ALLOC, LOAD, DATA
  3 .stabstr        00000001 f01020d1 001020d1 000030d1  2**0
    CONTENTS, ALLOC, LOAD, DATA
  4 .data           00009300 f0103000 00103000 00004000  2**12
    CONTENTS, ALLOC, LOAD, DATA
  5 .got            00000008 f010c300 0010c300 0000d300  2**2
    CONTENTS, ALLOC, LOAD, DATA
  6 .got.plt        0000000c f010c308 0010c308 0000d308  2**2
    CONTENTS, ALLOC, LOAD, DATA
  7 .data.rel.local 00001000 f010d000 0010d000 0000e000  2**12
    CONTENTS, ALLOC, LOAD, DATA
```

```

8 .data.rel.ro.local 00000044 f010e000 0010e000 0000f000 2**2
    CONTENTS, ALLOC, LOAD, DATA
9 .bss                00000661 f010e060 0010e060 0000f060 2**5
    CONTENTS, ALLOC, LOAD, DATA
10 .comment           0000002b 00000000 00000000 0000f6c1 2**0
    CONTENTS, READONLY

```

Part 3: The Kernel

Memory Mapping

```

~/oslab/lab$ objdump -f obj/kern/kernel

obj/kern/kernel:      file format elf32-i386
architecture: i386, flags 0x00000112:
EXEC_P, HAS_SYMS, D_PAGED
start address 0x0010000c

```

在BIOS进入boot loader时，物理地址0x00100000的内容是未初始化的内存。而在boot loader进入内核时，该地址的内容已经变成了内核的.text代码段的前几个字（见上面的Program Header输出条目）。地址0x00100000的内容在boot loader调用bootmain()函数之后，执行readseg()函数的过程中被改变的。

```

(gdb) x/8x 0x100000
0x100000:      0x00000000      0x00000000      0x00000000      0x00000000
0x100010:      0x00000000      0x00000000      0x00000000      0x00000000

(gdb) x/8x 0x100000
0x100000:      0x1badb002      0x00000000      0xe4524ffe      0x7205c766
0x100010:      0x34000004      0xd000b812      0x220f0010      0xc0200fd8

```

```

kern/kernel.ld

/* Link the kernel at this address: "." means the current address */
. = 0xF0100000;

/* AT(...) gives the load address of this section, which tells
   the boot loader where to load the kernel in physical memory */
.text : AT(0x100000) {
    *(.text .stub .text.* .gnu.linkonce.t.*)
}

```

Operating system kernels linked and run at very high virtual address, such as 0xf0100000

in order to leave the lower part of the processor's virtual address space for user programs to use

map virtual address 0xf0100000 (the link address at which the kernel code expects to run) to physical address 0x00100000 (where the boot loader loaded the kernel into physical memory)

Map the First 4MB of Physical Memory

Up until `kern/entry.S` sets the `CR0_PG` flag, memory references are treated as physical addresses.

Once `CR0_PG` is set, memory references are virtual addresses that get translated by the virtual memory hardware to physical addresses.

`entry_pgdir` translates virtual addresses

```
VA -> PA
0xf0000000 ~ 0xf0400000 -> 0x00000000 ~ 0x00400000
0x00000000 ~ 0x00400000 -> 0x00000000 ~ 0x00400000
other addresses -> hardware exception
```

```
obj/kern/kernel.s
```

```
f0100025:  0f 22 c0                mov    %eax,%cr0
```

在执行这一条指令之后，新的mapping已经建立了，`0xf0100000`会被映射到`0x00100000`

```
(gdb) b *0x0100025
Breakpoint 2 at 0x100025
(gdb) c
Continuing.
=> 0x100025:    mov    %eax,%cr0

Breakpoint 2, 0x00100025 in ?? ()
(gdb) x/8x 0x00100000
0x100000:      0x1badb002      0x00000000      0xe4524ffe      0x7205c766
0x100010:      0x34000004      0xd000b812      0x220f0010      0xc0200fd8
(gdb) x/8x 0xf0100000
0xf0100000 <_start-268435468>:  0x00000000      0x00000000      0x00000000      0x00000000
0x00000000
0xf0100010 <entry+4>:    0x00000000      0x00000000      0x00000000      0x00000000
(gdb) si
=> 0x100028:    mov    $0xf010002f,%eax
0x00100028 in ?? ()
(gdb) x/8x 0x00100000
0x100000:      0x1badb002      0x00000000      0xe4524ffe      0x7205c766
0x100010:      0x34000004      0xd000b812      0x220f0010      0xc0200fd8
(gdb) x/8x 0xf0100000
0xf0100000 <_start-268435468>:  0x1badb002      0x00000000      0xe4524ffe
0x7205c766
0xf0100010 <entry+4>:    0x34000004      0xd000b812      0x220f0010      0xc0200fd8
```

这是建立映射后的第一条指令

```
mov $relocated, %eax
```

注释kern/entry.S中的movl %eax, %cr0后

```
(gdb) b *0x100020
Breakpoint 1 at 0x100020
(gdb) c
Continuing.
The target architecture is set to "i386".
=> 0x100020:    or      $0x80010001,%eax

Breakpoint 1, 0x00100020 in ?? ()
(gdb) si
=> 0x100025:    mov     $0xf010002c,%eax <- first instruction
0x00100025 in ?? ()
(gdb) si
=> 0x10002a:    jmp     *%eax
0x0010002a in ?? ()
(gdb) si
=> 0xf010002c <relocated>:    add     %al,(%eax) <- fail to work
0xf010002c in relocated ()
(gdb) si
Remote connection closed
```

因为此时没有建立正确映射，导致链接地址杯当作物理地址访问

```
0xf010002c <relocated>: 0x00000000    0x00000000    0x00000000    0x00000000
```

对应加载出来的错误指令add %al,(%eax)，因为它的机器码为00 00

Formatted Printing to the Console

kern/printf.c, lib/printfmt.c, and kern/console.c,

kern/init.c调用cprintf函数，这是print.c暴露的接口

```
kern/init.c

cprintf("entering test_backtrace %d\n", x);
```

code completion

八进制无符号数的输出

```
lib/printfmt.c

// (unsigned) octal
case 'o':
    // Replace this with your code.
    num = getuint(&ap, lflag);
    base = 8;
    goto number;
    break;
```

Q&A

1. 下面看cprintf的实现

```
kern/print.c

int
cprintf(const char *fmt, ...)
{
    va_list ap;                // variable arguments
    int cnt;

    va_start(ap, fmt);        // 取出参数到ap中，并指定...之前的参数
    cnt = vprintf(fmt, ap);    // 将取出的参数列表传给真正的实现函数
    va_end(ap);               // 释放参数列表

    return cnt;
}

int
vprintf(const char *fmt, va_list ap)
{
    int cnt = 0;

    vprintfmt((void*)putch, &cnt, fmt, ap);
    return cnt;
}
```

printf.c调用了printfmt.c中的vprintfmt，它的一个参数为printf.c中的putch函数，而putch函数又调用了console.c中的cputchar函数。

所以cputchar是console.c暴露的接口。

```
cprintf -> vprintf -> vprintfmt -> putch -> cputchar
```

2. 解释以下代码

```
kern/console.c

// What is the purpose of this?
if (crt_pos >= CRT_SIZE) { // 检查屏幕是否已经写满
    int i;

    memmove(crt_buf, crt_buf + CRT_COLS, (CRT_SIZE - CRT_COLS) *
sizeof(uint16_t)); // 将第2行到最后1行的内容，整体向上移动一行
    for (i = CRT_SIZE - CRT_COLS; i < CRT_SIZE; i++)
        crt_buf[i] = 0x0700 | ' '; // 将屏幕的最后一行（新腾出来的空行）全部填充为空格
    crt_pos -= CRT_COLS; // 将光标位置向上移动一行，定位在新清空的最后一行开头
}
```

这段代码是控制台屏幕滚动的实现。当屏幕内容写满时，需要将屏幕向上滚动一行，为新行腾出空间。

3. 下面进入cprintf内部的执行过程

```
int x = 1, y = 3, z = 4;
cprintf("x %d, y %x, z %d\n", x, y, z);
```

fmt -> "x %d, y %x, z %d\n"

ap -> 第一个参数的地址

可变参数被连续地存储在栈上，可以从小到大增大索引访问到它们

存储的顺序为从最右边的参数开始压栈

```
高地址
z = 4
y = 3
x = 1 <- ap
fmt = "x %d, y %x, z %d\n"
低地址
```

接下来按顺序分析cons_putc, va_arg和vcprintf的调用

```
cprintf("x %d, y %x, z %d\n", x, y, z)
->
vcprintf("x %d, y %x, z %d\n", (char *)&fmt + sizeof(fmt)) // ap指向第一个参数x
->
vprintfmt((void*)putc, &cnt, fmt, ap)
->
putc('x', &cnt)
```

```

->
cputchar('x') // 120为'x'的ASCII码值
->
cons_putc('x')
->
相同步骤cons_putc(' ')
->
// (ch = *(unsigned char *) fmt++) == '%'
// case 'd'
getint(&ap, lflag=0)
->
va_arg(*ap, int)
// before: ap-> (x=1)
// after: ap-> (y=3)
->
printrnum(putch, putdat, num=1, base=10, width=-1, padc=' ')
->
putch('1', putdat)
->
cons_putc('1')
->
类似地打印后续字符串

```

4. 在kern/monitor.c中新增以下代码

```

int
mon_ans(int argc, char **argv, struct Trapframe *tf)
{
    unsigned int i = 0x00646c72;
    cprintf("H%x Wo%s", 57616, &i);
    return 0;
}
static struct Command commands[] = {
    { "testprintf", "Test cprintf", mon_testprintf },
};

```

```

K> ans
He110 World

```

因为57616的十六进制为0xe110

r, l, d的ASCII码分别为0x72, 0x6C, 0x64

所以按%x方式解析57616时输出e110

按%s方式解析0x00646c72（小端法）时输出rld

5. 如果执行

```
cprintf("x=%d y=%d", 3);
```

在输出到第2个%d时，`getuint`函数读取`ap`所指向的4个字节作为`num`

但只传入了1个可变参数，此时`ap`指向参数3之后的地址，包含随机的值

将其作为十六进制解析后输出，就是`y=`之后的输出

6. 逆序压栈参数

颠倒参数的压栈顺序，变为从左往右，应该修改`va_start`设置`ap`为第1个参数位置(`(ap) = (va_list)((char*)&fmt - sizeof(fmt))`)，修改`va_arg`取出下1个参数的方式为减小地址

The Stack

Initialization

```
kern/entry.S

movl    $(bootstacktop),%esp
->
    obj/kern/kernel.asm

f0100034:    bc 00 b0 10 f0        mov    $0xf010b000,%esp

.data
#####
# boot stack
#####
    .p2align    PGSHIFT    # force page alignment
    .globl      bootstack
bootstack:
    .space      KSTKSIZE
    .globl      bootstacktop
bootstacktop:
```

由反汇编可以看出栈顶位于`$0xf010b000`，`%esp`被设置为这个值，栈向低地址生长

内核通过`.space`在数据段中`bootstack`位置处初始化了`KSTKSIZE`这么多的空间

```
inc/memlayout.h

// All physical memory mapped at this address
#define KERNBASE    0xF0000000

// Kernel stack.
#define KSTACKTOP    KERNBASE
#define KSTKSIZE    (8*PGSIZE)    // size of a kernel stack
```

```
#define KSTKGAP      (8*PGSIZE)          // size of a kernel stack guard

inc/mmu.h

#define PGSIZE      4096                // bytes mapped by a page
```

Stack Backtrace

%esp 指针指向栈顶，即栈被使用部分的最低地址，随着 **push** 和 **pop** 指令动态地变化

%ebp 指针指向当前函数栈帧的基地址，对于追溯非法参数的传入非常关键

```
obj/kern/kernel.asm

f0100040 <test_backtrace>:
```

```
(gdb) b test_backtrace
Breakpoint 1 at 0xf0100040
(gdb) c
Continuing.
The target architecture is set to "i386".
=> 0xf0100040 <test_backtrace>: push    %ebp

Breakpoint 1, 0xf0100040 in test_backtrace ()
(gdb) x/8wx $esp
0xf010afd8: 0xf01000f4 0x00000005 0x00001aac 0x00000660
0xf010afe8: 0x00000000 0x00000000 0x00010094 0x00000000
(gdb) si
=> 0xf0100041 <test_backtrace+1>: mov    %esp,%ebp
0xf0100041 in test_backtrace ()
(gdb) x/8wx $esp
0xf010afd8: 0xf010aff8 0xf01000f4 0x00000005 0x00001aac
0xf010afe8: 0x00000660 0x00000000 0x00000000 0x00010094
(gdb) si
=> 0xf0100043 <test_backtrace+3>: push   %esi
0xf0100043 in test_backtrace ()
(gdb) x/8wx $esp
0xf010afd8: 0xf010aff8 0xf01000f4 0x00000005 0x00001aac
0xf010afe8: 0x00000660 0x00000000 0x00000000 0x00010094
(gdb) si
=> 0xf0100044 <test_backtrace+4>: push   %ebx
0xf0100044 in test_backtrace ()
(gdb) x/8wx $esp
0xf010afd4: 0x00010094 0xf010aff8 0xf01000f4 0x00000005
0xf010afe4: 0x00001aac 0x00000660 0x00000000 0x00000000
```

```
0xf010afbc: 0xf0100076    <- ESP指向这里 (递归返回地址)
0xf010afc0: 0x00000004
0xf010afc4: 0x00000005
0xf010afc8: 0x00000000
0xf010afcc: 0xf010004a
0xf010afd0: 0xf010c308
0xf010afd4: 0x00010094
0xf010afd8: 0xf010aff8    <- push    %ebp
0xf010afdc: 0xf01000f4    <- 压入返回地址i386_init到旧的ESP
0xf010afe0: 0x00000005    <- 参数 x=5 (第一层)
0xf010afe4: 0x00001aac
0xf010afe8: 0x00000660
0xf010afec: 0x00000000
0xf010aff0: 0x00000000
0xf010aff4: 0x00010094
0xf010aff8: 0x00000000    <- 第一层的EBP (栈底)
```